library system

CS342

Sections: 5303 / 5298

students names:

Sana Al-Edilbi
Safia Jalal
Leena ALmutairi
Zainab Komailah
Raghad Alobaid

Lecturer name: Rafif Alsuhaibani

1-Application overview

This application is a Library Management System that allows users to borrow and reserve books.

To access the system, users can either log in using their email and password or sign up as new members to create an account. The system also includes an Admin account, which is pre-registered in the database.

The admin has full management privileges, including:

- Adding new books
- Editing existing book details
- Deleting books from the system
- Viewing all registered members
- Checking borrowing and reservation activities

The Member account provides access to browse and manage library transactions include:

- Search for books by **Book ID**, **Title**, **Author**, **ISBN**, or **Category**. Upon selecting a book, they can view detailed information, including the **total number of copies** and the current **status of each copy** (Available, Checked Out, or Reserved).
- Easily borrow available copies and process returns for books currently checked out.
- Extend the loan period for borrowed items, provided the book has not been reserved by another member.
- Reserve books that are currently out of stock or checked out. The system places the user in a queue and notifies them when a copy becomes available.

2-GUI elements

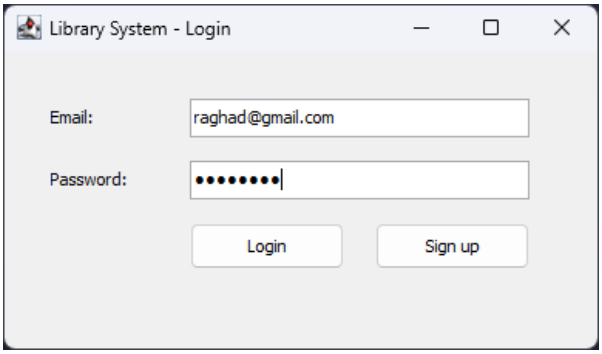
the interface was implemented using java swing library,‘clear and user-friendly’

The user interface is built using Java Swing library to ensure it is simple and easy to navigate. The design uses a tabbed layout, which allows users to switch between different screens, such as searching for books or managing loans, without leaving the main window. The application starts with a secure login screen that automatically directs the user to the correct dashboard based on their role. Members see a simple view for borrowing and returning items, while Librarians access a full control panel for managing the system. All data, such as book lists and user details, is displayed in clear tables that update immediately after any change, allowing users to interact with the system smoothly.

2.1 Authentication

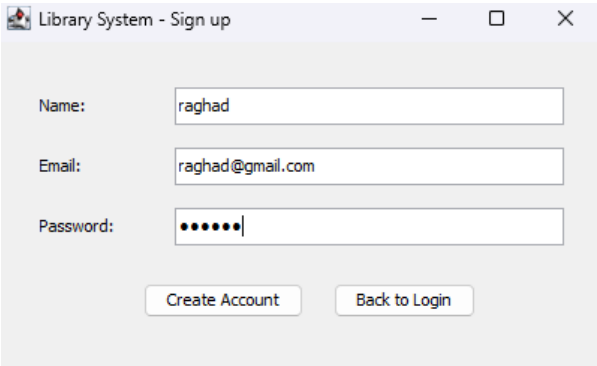
2.1.1 Login page

The login page has a simple user-friendly design.



- Registered users: The user enters their account information then clicks the login button to access their account.then it either opens a librarian admin page or member page.

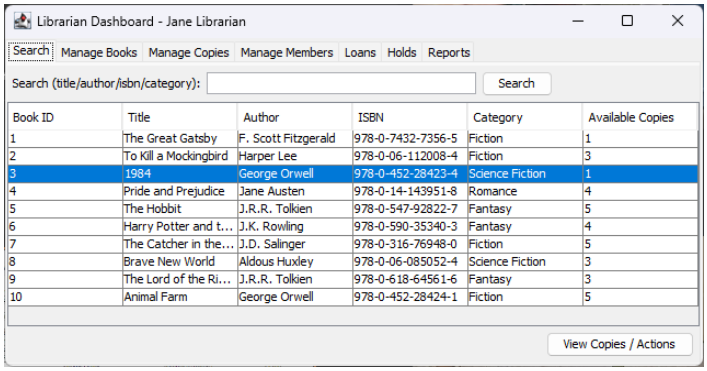
2.1.2 Sign up page



- New users: for new users they need to create an account to login, so by pressing “sign up” button it will switch the window to sign up window ,as shown in figure 2.2, then they need to fill their informations name, email and password

2.2 Dashboards

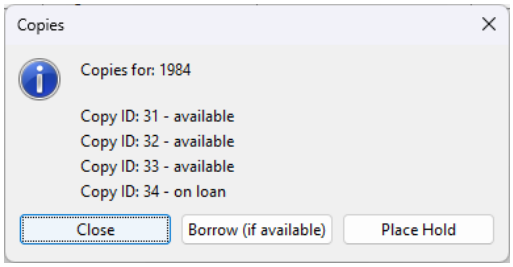
2.2.1 Librarian dashboard/Admin page



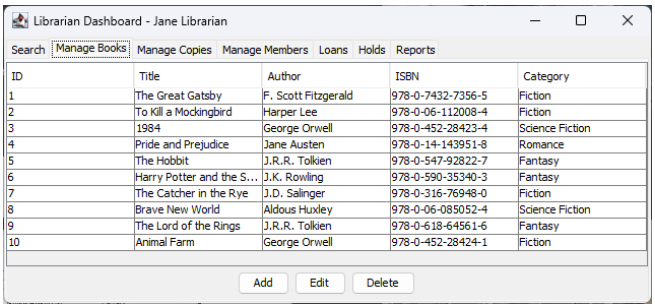
This page is for admins only/ this page only admins can access it.

Page content:

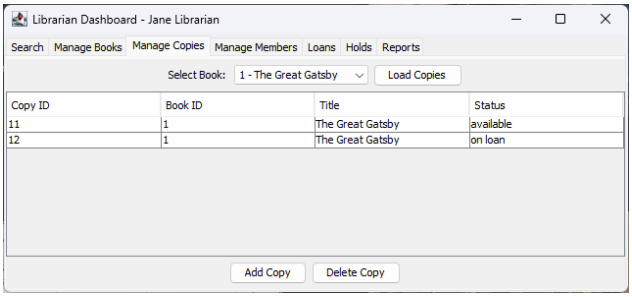
- navigation bar: admins can easily navigate the system tabs with this bar.
  - Search tap:
    - Search box:admin enters book attribute to search of certain books .
    - Tables area: the search result appears as a table content.
    - Action button: pops up a window to show copies of the selected book and actions buttons, as:
      - Closing the popup window.
      - Borrow book.
      - Place a hold on a book.



  - Manage book tab:
    - Table: all books are shown in this table with all their attributes as title, author, N.O of copies and so on.
    - Add new button: adding new book.
    - Edit button: to edit existing book details or fixing errors, like titles and author name.
    - Delete button: to delete a book if it's no longer existing in the library.

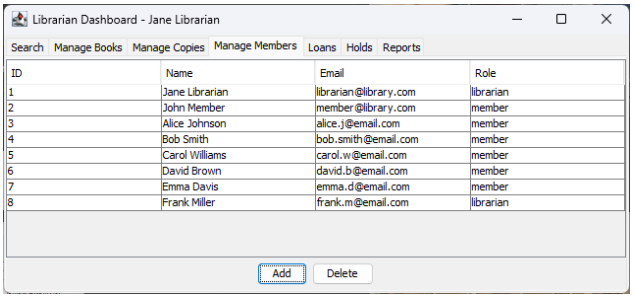


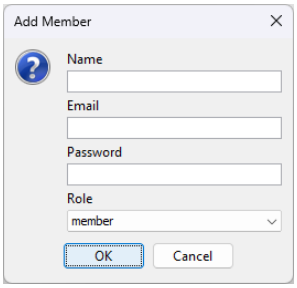
- Manage copies tab:
  - Select book list: admin choose the book that they need.
  - Loading table: by pressing the “load copies” button, the selected book will appear as a table with details, like book ID, copy ID and status if it’s available, on hold or on loan.



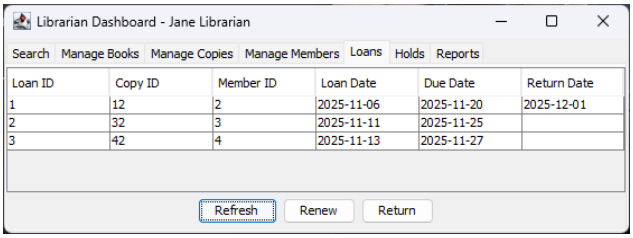
  - Add and delete copies: adding a new copy button will add a new copy of the selected book with a new ID and mark the status as available.

- Manage members tab:
  - Members information table: all members information will be shown and their attributes, like ID, account information and role.
  - Add and delete members buttons: if adding members button clicks a popup window will appear containing fields to fill new member information and role.





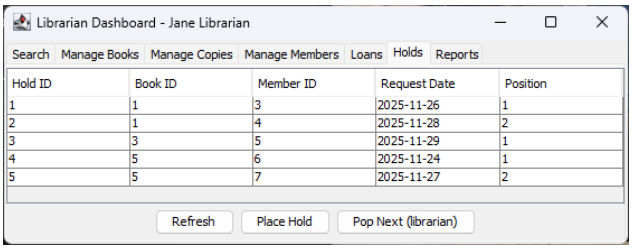
- Loans tab:
  - Table:loans data will be shown in the table with their attributes, loan ID, copy ID, member ID, date of the loan, the due date and return date.



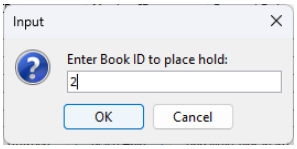
  - Refresh button: refreshes the page to show the updates that were made.
  - Renew button: renews existing loan upon user request for renewal and updates the loan date.

- Return button: records a return date on selected loan.

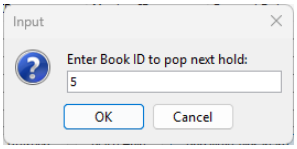
- Holds tab:
  This tab has all the content of existing hold requests on books, it contains:



  - Table:holds data will be shown in the table with their attributes, hold ID, book ID, member ID, request date, and position.

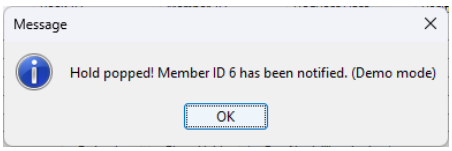


  - Refresh button: refreshes the page to show the updates that were made.

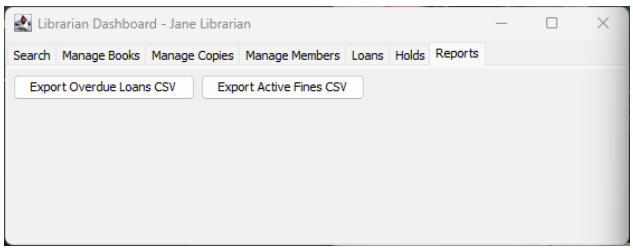


  - Place hold button: pops up a window for adding a new hold on a book by adding a book ID.

  - Pop next button: notifies the member who requested a hold on a book that it's now available.



- Reports tab:
  This tab contains CSV report generators, can be triggered with these buttons:



  - Export overdue loans button: clicking this button will export the data of the users who didn't return their books on time in CSV form.

```
loan_id,copy_id,member_id,due_date
1,12,2,2025-11-21
2,32,3,2025-11-26
3,42,4,2025-11-28
```

  - Export active fines button: clicking this button will export the data of the active fines on members, their information and fine amount in CSV form.

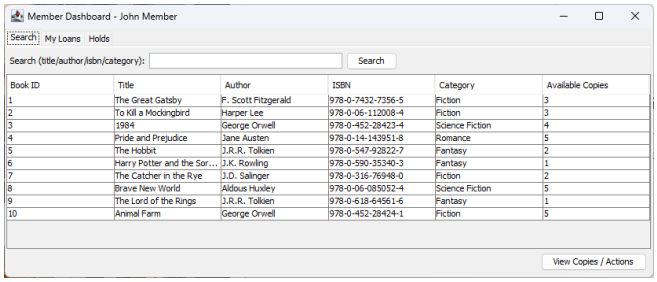
```
fine_id,member_name,member_email,amount
1,John Member,member@library.com,5.50
2,Alice Johnson,alice.j@email.com,3.00
3,Bob Smith,bob.smith@email.com,12.75
```

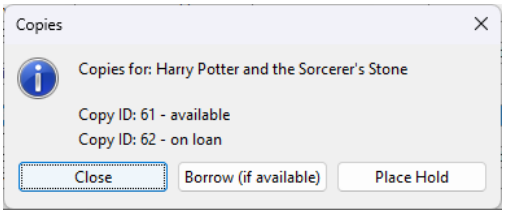
2.2.2 Member Dashboard/User dashboard

This page can be accessed by library members, its easy to navigate and users can see their book loans and hold easily with this dashboard.

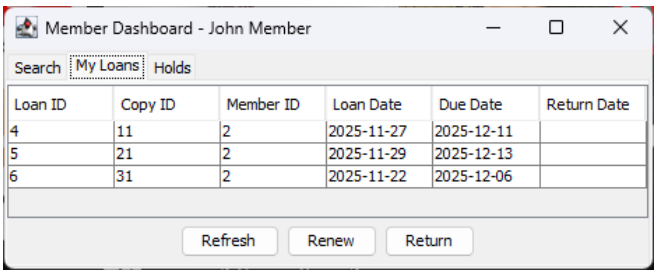
Windowcontent:

- Navigation bar: using this bar users can switch between taps so easily. Bar contains:
- Search tap:
  - Search box:admin enters book attribute to search for certain books .
  - Tables area: the search result appears as a table content.
  - Action button: pops up a window to show copies of the selected book and actions buttons, as:
    - Closing the popup window.
    - Borrow book.
    - Place a hold on a book.



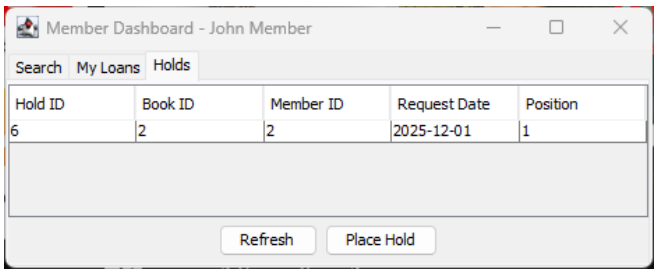


- My loans tab:
  - Table:loans data will be shown in the table with their attributes, loan ID, copy ID, member ID, date of the loan, the due date and return date.
  - Refresh button: refreshes the page to show the updates that were made.
  - Renew button: renews existing loan upon user request for renewal and updates the loan date.
  - Return button: records a return date on selected loan.

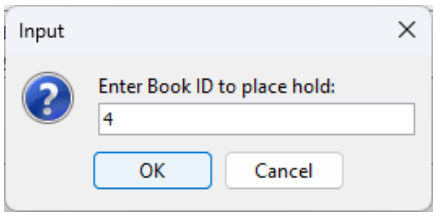


- Holds tab:

  This tab has all the content of existing hold requests on books, it contains:
  - Table:holds data will be shown in the table with their attributes, hold



ID, book ID, member ID, request date, and position.
- Refresh button: refreshes the page to show the updates that were made.
- Place hold button: pops up a window for adding a new hold on a book by adding a book ID.



3-Database connection

3.1 Overview

The application system integrate the database connectivity using the JDBC API with MySQL as the backend database. the connector/j dependency is added directly inside the pom.xml file, it enables the connection between the app and MySQL server.

Then created a database connection class,which is responsible for handling connections.

3.2 Connection configuration:

- Database management system used: MySQL
- The database URL: jdbc:mysql://localhost:3306/library
- Username and password

3.3 key methods

- getConnection(): returns active database connection.
- close(): properly closes the connection when the app is closed
- A try–catch blocks to handle connection errors.

3.4 Database Schema:

Tables:

- members: member_id, name, email, password_hash, role
- books: book_id, title, author, isbn, category, description
- copies: copy_id, book_id, status
- loans: loan_id, copy_id, member_id, loan_date, due_date, return_date
- holds: hold_id, book_id, member_id, request_date, position
- fines: fine_id, member_id, amount, is_paid

After the connection is done, the system can:

- Allow admin to Add, edit, and delete books
- Register new members
- Validate login information
- Save borrowing and reservation data
- Allow admin to view and manage members.
- Allow members to view and manage their books, loans and holds.

4-Exception handling

The system implements exception handling at two layers: data access (DAO) and user interface (UI).

4.1 exceptions used

- SQL exceptions (java.sql.SQLException)
  - Most common and handles all database related errors.
  - Thrown by all DAO methods, caught by UI.
- IO exceptions (java.io.IOException)
  - Used by reportPanal for report generation.
  - Handles file writing errors for CSV export.
- NoSuchAlgorithmException (java.security.NoSuchAlgorithmException)
  - Handles missing SHA-256 hashing algorithm.
  - Wrapped in RuntimeException in PasswordHasher class.

4.2 Exception Handling Strategy

- DAO Layer:
  - All methods declare “throws SQLException”.
  - Exceptions propagated to the UI layer.
  - Try-with-resources used for automatic cleanup.

- UI Layer:
  - Catches exceptions from DAO methods.
  - Displays error dialogs using JOptionPane.
  - Logs stack traces to console with printStackTrace().

4.3 scenarios

- Database Connection Failures:
  - Exception:SQLException
    - Database server not running
    - Incorrect credentials
  - User sees: "Database error: [message]".

- Authentication Errors
  - Exception: SQLException:
    - Wrong email/password (returns null, not exception).
    - Database unavailable during login.
  - User sees: "Invalid credentials" or "Database error: [message]".

- Registration Errors:
  - Exception: SQLException
    - Duplicate email registration
    - Database errors during account creation
  - User sees: "Email already registered" or "Error: [message]".

- Input Validation Errors:

  No Exception Thrown; Handled with if-statements before database calls
  - Scenarios:
    - Empty fields (name, email, password)
    - Invalid email format (regex check)
    - Weak password (less than 6 characters)
    - Password complexity requirements not met
  - User Messages:
    - "All fields required"
    - "Please enter a valid email address”
    - "Password must be at least 6 characters long"
    - "Password must include uppercase, lowercase, number, and special characters".

- Book Management Errors
  - Exception: SQLException
  - Scenarios:
    - Adding/updating/deleting books fails.
    - Book not found (returns null).
    - Deleting book with existing copies (foreign key constraint).
    - Search operation fails.
  - User Messages:
    - "Select a book" (no selection made).
    - Error: [SQL message]".
    - "Search error: [message]".

- Copy Management Errors
  - Exception: SQLException
  - Scenarios:
    - Adding/deleting copies fails.
    - No book selected when adding or deleting a copy.
  - User Messages:
    - "Select a book".
    - "Select a copy".
    - "Error: [message]".

- Loan Management Errors
  - Exception:SQLException
  - Scenarios:
    - Loan not found (custom SQLException thrown).
    - Renewing overdue loan (custom SQLException thrown).
    - Renewing loan with holds (custom SQLException thrown).
    - No available copies to borrow (returns null)
    - No loan selected for return/renew
  - User Messages:
    - "Select a loan to return/renew".
    - "No available copies. You can place a hold".
    - "Cannot renew: Cannot renew overdue loan".
    - "Cannot renew: holds exist for this book".
    - "Only members can borrow".

- Hold Management Errors
  - Exception:SQLException, NumberFormatException
  - Scenarios:
    - Invalid book ID entered (non-numeric).
    - Hold placement fails - No holds available to pop.
    - Librarian attempting member-only actions
  - User Messages:
    - "Invalid ID".
    - "Error: [SQL message]".
    - "No holds" - "Only members can place holds".

- Member Management Errors
  - Exception: SQLException
  - Scenarios:
    - No member selected for deletion.
    - Deleting member with active loans/fines (constraint violation).
    - Adding member fails.
  - User Messages:
    - "Select a member".
    - "Error: [message]".

- Report Generation Errors
  - Exception: SQLException, IOException
  - Scenarios:
    - Database query fails when fetching data.
    - Cannot write CSV file to disk.
    - File permission denied.
    - Disk space insufficient
  - User Messages:
    - "Error: [message]".

- Password Hashing Error
  - Exception: NoSuchAlgorithmException (wrapped in RuntimeException).
  - Scenario:
    - SHA-256 algorithm not available in JVM (extremely rare).
    - Application would crash if this occurs.
    - Wrapped in RuntimeException because it's a critical failure.

5-Testing and validation

The application was tested manually through the user interface to ensure all features work correctly and handle errors appropriately.

5.1 Testing Approach

- Manual Testing:
  - Each feature was tested by entering data through the UI forms.
  - Database state was verified after operations.
  - Error messages were checked for clarity.
  - Both normal operations and edge cases were tested.

- Test Environment:
  - Database: MySQL.
  - JDK Version: 8+.
  - UI Framework: Java Swing.

5.2 Input Validation

The system implements validation at the input level to prevent invalid data from being processed.

5.2.1 Email Validation

- Implementation: Regex pattern matching in SignupFrame.java.
- Pattern Used: “^[\w.%+-]+@[\w.-]+\.[A-Za-z]{2,6}$”.
- Validates:
  - Proper email format (user@domain.extension).
  - Rejects emails without @ symbol.
  - Rejects emails without domain extension.
  - Rejects emails with spaces or invalid characters.

- Error Message: "Please enter a valid email address".

5.2.2 Password Validation

- Implementation: Multiple security checks in SignupFrame.java.
- Requirements Enforced:
  - Minimum length of 6 characters.
  - At least one uppercase letter (A-Z).
  - At least one lowercase letter (a-z).
  - At least one number (0-9).
  - At least one special character (@$!%*?&).

- Pattern Used: “^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{6,}$”.

- Error Messages:
  - "Password must be at least 6 characters long".
  - "Password must include uppercase, lowercase, number, and special character".

5.2.3 Empty Field Validation

- Implementation: Checked before processing in all forms.
- Validates:
  - All required fields are filled.
  - Prevents database operations with incomplete data.
- Error Message: "All fields required".

5.3 Features Tested

- Authentication System
  - User login with valid/invalid credentials
  - New user registration
  - Duplicate email prevention
  - Password hashing (SHA-256)
  - Role-based access (Librarian vs Member)

- Book Management
  - Adding new books to catalog
  - Editing existing book information
  - Deleting books (with constraint checks)
  - Searching books by title, author, ISBN, category
  - Viewing all books in system

- Copy Management
    - Adding multiple copies of books
    - Deleting copies
    - Viewing copy status (available, checked_out, reserved)
    - Loading copies for specific books.

- Loan Management
    - Borrowing available books
    - Returning borrowed books
    - Renewing active loans
    - Preventing overdue loan renewals
    - Preventing renewals when holds exist
    - Fine calculation for overdue returns
    - Viewing member loans and overdue loans.

- Hold Management
    - Placing holds on books.
    - Queue position tracking.
    - FIFO (First In, First Out) processing.
    - Popping next hold in queue.
    - Position reordering after hold removal.
    - Hold notification system.

- Member Management
    - Adding new members (librarian/member roles).
    - Deleting members (with constraint checks).
    - Viewing all members in system.

- Report Generation
    - Exporting overdue loans to CSV.
    - Exporting active fines to CSV.
    - File creation and write operations.

- Fine System
    - Automatic fine calculation (1.0 per day overdue).
    - Fine tracking for members.
    - Unpaid fine reporting.

5.3.1 Error Handling Testing

- Database Errors:
  - Connection failures handled gracefully.
  - SQL errors caught and displayed to user.
  - Foreign key constraint violations handled.
- Input Errors:
  - Invalid email format rejected before database access
  - Weak passwords rejected before account creation
  - Empty fields caught at validation layer
- File Operations:
  - File write failures caught and reported
  - Permission errors handled
- Business Logic Errors:
  - Role-based restrictions enforced (member-only operations)
  - Loan renewal rules enforced
  - Hold system constraints validated

All errors display clear messages to users and log details to console for debugging.

5.3.2 Database Integrity Testing

- Constraint Enforcement:
  - Foreign key constraints prevent orphaned records.
  - Unique email constraint prevents duplicate accounts.
  - Referential integrity maintained across all tables.
- Transaction Testing:
  - Borrow operation updates copy status and creates loan.
  - Return operation updates copy status and records return date.
  - Hold pop operation removes hold and reorders queue.

5.3.3 User Interface Testing

- Navigation:
  - Tab switching between all panels.
  - Data persistence when switching views.
  - Table displays and refresh correctly.
- Forms:
  - Input fields accept valid data.
  - Validation messages display clearly.
  - Success/error dialogs appear appropriately.
- Tables:
  - Data displays correctly in all tables
  - Row selection works properly
  - Table updates after operations

5.3.4 Security Testing

- Password Security:
  - Passwords hashed using SHA-256.
  - Plain text passwords never stored.
  - Hash consistency verified (same password = same hash).
- Authentication:
  - Invalid credentials rejected.
  - Session management (user object passed to dashboard).
  - Role-based feature access enforced.

5.3.5 Testing Results

- Areas Tested:
  - Input validation (email, password, empty fields)
  - Authentication and authorization
  - All CRUD operations (Create, Read, Update, Delete)
  - Database connection and error handling
  - Business logic and constraints
  - User interface functionality
  - Report generation
  - Security (password hashing)

- Validation Implementation:
  - Email validation using regex pattern matching
  - Password validation with multiple security criteria
  - Empty field validation before processing
  - Business rule validation at DAO layer

Outcome:

All core features function correctly. Input validation prevents invalid data entry. Error handling ensures application stability. Security measures (password hashing, role-based access) work as intended. The system successfully handles both normal operations and error conditions.

Task Distribution Table :

Safia Jalal	Writing the program code
Sana Al-Edilbi	Assisting in coding and fixing errors
Leena Almutairi	Designing the database
Zainab Komailah	Inserting and testing data
Raghad Alobaid	Writing and formatting the report